2026 Spring – UAV Control Special Topics : Homework 4

Intelligent Vehicle & Mechatronics lab Advisor : Prof. Kang Li Author : Chih-Feng Huang

1. Learning to Use an Autopilot System (PX4 Offboard Control)

Objective

The purpose of this assignment is to help students understand the architecture of a modern UAV software stack and learn how ROS2, PX4, Gazebo, and QGroundControl interact with each other.

After completing this assignment, students should be able to:

- Launch PX4 SITL and Gazebo
- Connect and monitor the vehicle using QGroundControl
- Understand the roles of MAVLink, DDS, and uORB
- Develop a ROS2 node for autonomous UAV control
- Use PX4 Offboard Mode to execute autonomous flight tasks

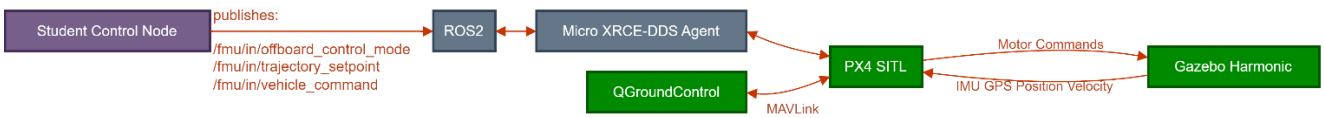


Fig1. Software Architecture

Assignment Description

Develop a ROS2 C++ node named: HoverController

The node must autonomously perform the following sequence:

1. Switch to Offboard Mode
2. Arm the UAV
3. Take off to 3 meters
4. Hover for 30 seconds
5. Land safely

Provided Environment

The following software has already been installed and configured:

- Ubuntu 22.04
- ROS2 Humble
- Gazebo Harmonic
- PX4 SITL
- QGroundControl
- MicroXRCEAgent

Available Topics

Publishers

- /fmu/in/offboard_control_mode
- /fmu/in/trajectory_setpoint
- /fmu/in/vehicle_command

Subscribers

- /fmu/out/vehicle_status_v4
- /fmu/out/vehicle_local_position_v1

Program Execution Logic

The program does not control Gazebo directly. Instead, the ROS2 node sends commands to PX4(High Level Command), and PX4 controls the UAV model in Gazebo.

Step

1. Start Gazebo + PX4 SITL
2. Start Micro XRCE-DDS Agent
3. Start QGroundControl
4. Run student ROS2 node
5. ROS2 node publishes Offboard commands
6. PX4 receives commands and controls the UAV
7. Gazebo simulates UAV motion

Execution Commands

Terminal 1: PX4 SITL + Gazebo

```
cd ~/PX4-Autopilot
make px4_sitl gz_x500
```

Terminal 2: Micro XRCE-DDS Agent

```
MicroXRCEAgent udp4 -p 8888
```

Terminal 3: QGroundControl

```
./QGroundControl-x86_64.AppImage
```

Terminal 4: Student Node

```
ros2 run hover_controller hover_node
```

Programming Requirements

You must use:

- rclcpp
- px4_msgs

You may NOT!!!:

- Use QGroundControl buttons to take off
- Use manual joystick control

The entire flight mission must be executed through your ROS2 node.

Flight Requirements

The UAV must:

- Successfully take off
- Hover for 30 seconds
- Reach 3.0 ± 0.3 m altitude
- Land safely

Student Node Logic

1. Continuously publish OffboardControlMode
2. Continuously publish TrajectorySetpoint
3. Send command to switch PX4 into Offboard Mode
4. Send command to arm the UAV
5. Monitor vehicle_local_position
6. Check whether altitude reaches 3 m
7. Maintain hover for 30 seconds
8. Send land command

2. Quadrotor Hover Control in Gazebo

Objective

In this assignment, you will implement your own altitude controller for a quadrotor in Gazebo.

Unlike HW1, where PX4 handled the low-level flight control, this assignment removes the PX4 controller entirely. You will directly command the motor speeds of the quadrotor and maintain a stable hover at a specified altitude.

The goal is to understand:

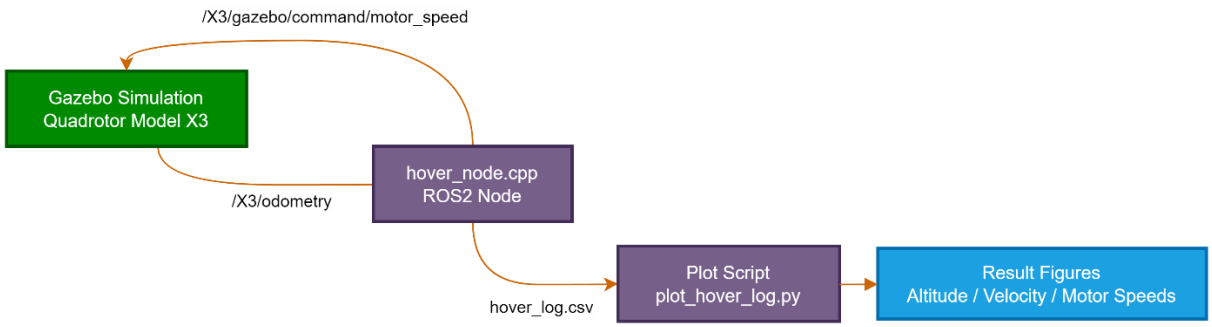
- Quadrotor dynamics
- Sensor-based state estimation
- Performance evaluation
- Feedback control
- Real-time control loops

Learning Outcomes

After completing this assignment, students should be able to:

- Read sensor data from Gazebo
- Design a feedback controller
- Analyze control performance
- Obtain altitude and vertical velocity information
- Generate motor commands
- Evaluate hover stability

Architecture



Assignment Task

Design a controller that enables the quadrotor to:

1. Take off from the ground
2. Reach a target altitude of 3.0 m
3. Maintain hover for 30 seconds

The controller must operate using sensor feedback only.

Provided Files

The following files will be provided:

uav_gazebo_hw/

```
├── hover_node.cpp
├── worlds/
│   └── hover_world.sdf
├── models/
│   └── simple_quadrotor/
├── scripts/
│   └── setup_env.sh
└── tools/
    └── plot_hover_log.py
```

hover_node.cpp

The provided node already implements:

- ROS2 publishers
- ROS2 subscribers
- Gazebo communication
- Data logging
- Safety handling

Students only need to complete the controller section.

All other code should remain unchanged.

Available Measurements

The node subscribes to: /X3/odometry

From this message you can access:

Variable	Description
z	Current altitude [m]
V_z	Vertical velocity [m/s]

Control Input

The controller publishes: /X3/gazebo/command/motor_speed

Output: omega0, omega1, omega2, omega3 (rad/s)

Any control strategy is allowed. There is no restriction on controller design.

Running the Simulation

Terminal 1 : Load the environment.

```
cd ~/uav_gazebo_hw
source scripts/setup_env.sh
```

```
start_gazebo
```

Terminal 2：Start the ROS-Gazebo bridge.

```
cd ~/uav_gazebo_hw
source scripts/setup_env.sh

start_bridge
```

Terminal 3：Compile and run your controller.

```
cd ~/your_ros2_ws
colcon build
source install/setup.bash
ros2 run hover_controller hover_node
```

Data Logging

The provided node automatically generates:

hover_log.csv

The log contains: time, z ,V_z, motor1, motor2, motor3, motor4

No additional logging implementation is required.

Plotting Results

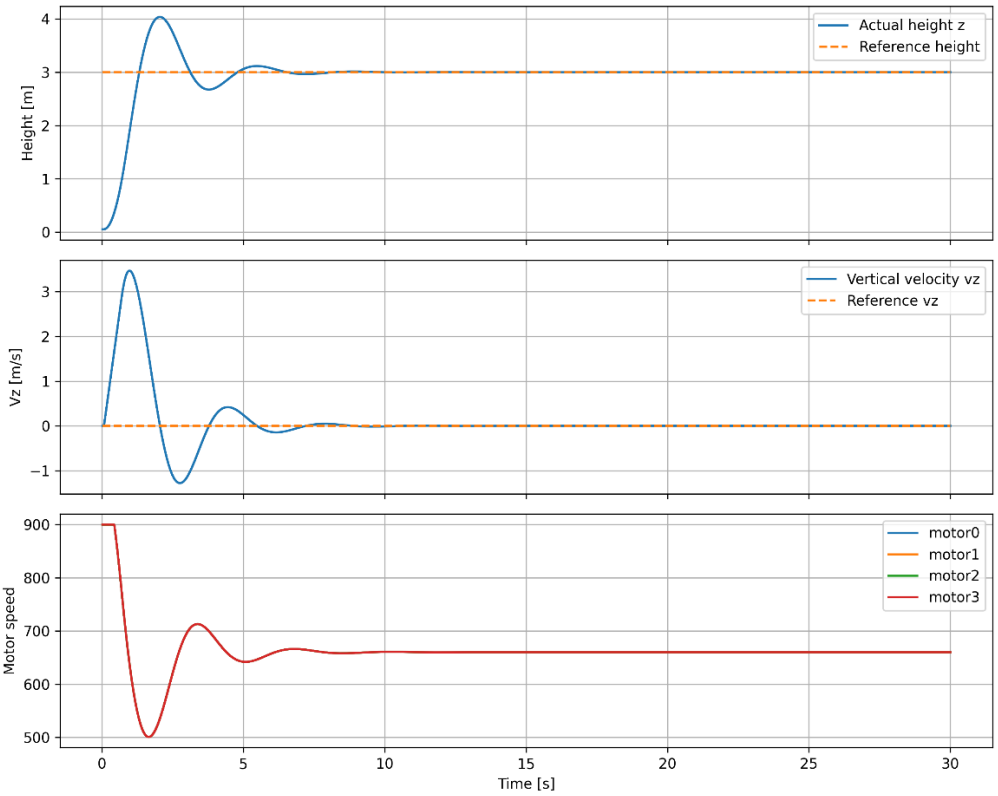
A plotting script is provided:

cd ~/uav_gazebo_hw/tools

python3 plot_hover_log.py

The script automatically generates: hover_result.png

EX:



The controller should minimize: Overshoot, Oscillation, Steady-state error

Report

1. Controller Design
- Control structure
- Design rationale
- Controller equations
2. Experimental Results
- Altitude plot
- Vertical velocity plot
- Motor speed plot
3. Discussion
- Hover performance
- Strengths and weaknesses
- Possible improvements